

PROYECTO GUÍA FULLSTACK

Technical Support Ticket Management System

MÓDULO 5 — Sistema de Roles y Seguridad (Extensión)

Objetivo: Implementar roles jerárquicos y restringir el acceso a los datos según la identidad del usuario.

En este **MÓDULO 5**, se implementará la mejora en la aplicación donde "todos pueden hacer todo" a un sistema empresarial con **Control de Acceso Basado en Roles (RBAC)**. Aprovechamos que la plantilla ya incluye **ASP.NET Core Identity** para extenderla, definiendo quién es el jefe (Admin), quién resuelve (Technician) y quién reporta (Employee).

Control de Acceso Basado en Roles (RBAC)

En **Visual Studio 2026**, el Control de Acceso Basado en Roles (**RBAC**) no es una función interna del editor de código en sí, sino un modelo de seguridad que los desarrolladores implementan en sus aplicaciones y que Microsoft integra en sus servicios en la nube (como Azure) para gestionar quién puede ver o modificar recursos.

Fundamentos del RBAC

- **Asignación de Roles:** En lugar de dar permisos individuales a cada persona, se crean roles (por ejemplo: "Administrador", "Técnico", "Empleado", "Cliente") con permisos específicos.
- **Principio de Privilegio Mínimo:** Garantiza que los usuarios solo tengan el acceso estrictamente necesario para cumplir su función, reduciendo riesgos de seguridad.
- **Entidades de Seguridad:** Los roles se pueden asignar a usuarios individuales, grupos de seguridad o identidades administradas de aplicaciones.

RBAC en el Flujo de Desarrollo (Visual Studio 2026)

- **Implementación en Aplicaciones:** Los desarrolladores usan Visual Studio 2026 para definir roles dentro del código (por ejemplo, mediante atributos como `[Authorize(Roles = "Admin")]` en ASP.NET Core) para controlar el acceso a secciones específicas de una aplicación.
- **Gestión de Recursos de Azure:** Si utilizas Visual Studio para desplegar en la nube, el RBAC de Azure determina tus capacidades (por ejemplo si puedes crear bases de datos o solo ver logs de errores).
- **Seguridad Unificada:** En 2026, Microsoft ha impulsado un "RBAC Unificado" que permite gestionar permisos de diferentes servicios (como Microsoft Sentinel o Defender) desde un solo panel, simplificando la administración para equipos de desarrollo y operaciones.

Beneficios

1. **Simplicidad Administrativa:** Facilita la gestión de equipos grandes; si un desarrollador cambia de puesto, solo se le cambia el rol en lugar de actualizar decenas de permisos individuales.
2. **Cumplimiento y Auditoría:** Permite rastrear fácilmente quién tiene acceso a datos sensibles, algo crítico para normativas de protección de datos.
3. **Jerarquías de Roles:** Los roles pueden heredar permisos de niveles inferiores, lo que permite una estructura de seguridad organizada y escalable.

1. Definición de Roles Constant

Para evitar errores al escribir nombres de roles en los atributos, crearemos una clase de constantes estáticas.

Archivo: /Data/Roles.cs

```
3 | public static class Roles
4 | {
5 |     public const string Admin = "Admin";
6 |     public const string Technician = "Technician";
7 |     public const string Employee = "Employee";
8 | }
9 |
```

2. Inicialización de Roles y Super-Usuario (Seeding)

Debemos asegurar que los roles existan en la base de datos SQL Server apenas inicie la aplicación. Modificar el bloque de inicialización creado en el Módulo 4 dentro de Program.cs.

Actualización en Program.cs, editar:

- **Importante sobre IdentityRole: Asegurarse** de tener este using en la parte superior de Program.cs: `using Microsoft.AspNetCore.Identity;`
- **.AddRoles<IdentityRole>()**: Registra formalmente el servicio RoleManager en el contenedor de dependencias. Sin esto, cuando intentas pedir el RoleManager en el bloque de *Seeding*, el sistema no lo reconoce y se produce una excepción.
- **IdentityRole**: Es la clase estándar de Microsoft para manejar los nombres de los roles en la base de datos.

```
33 | // VERIFICAR Y EDITAR EL SERVICIO DE IDENTIDAD PARA QUE REQUIERA CONFIRMAR CUENTA ANTES DE PERMITIR INICIAR SESIÓN
34 | builder.Services.AddIdentityCore<ApplicationUser>(options =>
35 |     {
36 |         options.SignIn.RequireConfirmedAccount = true;
37 |         options.Stores.SchemaVersion = IdentitySchemaVersions.Version3;
38 |     })
39 |     .AddRoles<IdentityRole>()
40 |     .AddEntityFrameworkStores<ApplicationDbContext>()
41 |     .AddSignInManager()
42 |     .AddDefaultTokenProviders();
43 |
44 | builder.Services.AddSingleton<IEmailSender<ApplicationUser>, IdentityNoOpEmailSender>();
```

Registro de los cambios en el control de Versiones

```
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   SupportTicketsSystem2026/Program.cs

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    SupportTicketsSystem2026/Data/Roles.cs

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   SupportTicketsSystem2026/Data/Roles.cs
    modified:   SupportTicketsSystem2026/Program.cs

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Definición de Roles Constantes"
[main 7ae5e4d] Definición de Roles Constantes
 2 files changed, 9 insertions(+)
 create mode 100644 SupportTicketsSystem2026/Data/Roles.cs
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```

Para pedir servicios (como `RoleManager`), necesitamos que el contenedor de dependencias esté listo.

Antes del `app.Run()`: Queremos que la base de datos esté lista y con los roles creados **antes** de que el primer usuario intente entrar.

Contexto de Scope: Usamos `CreateScope()` porque el `DbContext` y los Managers son servicios "Scoped". Esto asegura que, una vez terminemos de crear los roles, esos recursos se liberen correctamente de la memoria.

Observación:

Como estamos en **.NET 10, C# 14**, notar la sintaxis de **Collection Expressions** para los roles: `string[] roleNames = [Roles.Admin, Roles.Technician, Roles.Employee];` (usando corchetes `[]` en lugar de llaves `{}`). Continuamos con **program.cs**:

```
67
68 // Agregamos los endpoints de Identity para el manejo de autenticación y autorización
69 app.MapAdditionalIdentityEndpoints();
70
71 // =====
72 // BLOQUE DE SEEDING (INICIO): en modo desarrollo,
73 // se asegura de que existan los roles, un usuario
74 // admin por defecto y algunas categorías para tickets.
75 // =====
76 using (var scope = app.Services.CreateScope())
77 {
78     var services = scope.ServiceProvider;
79     var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
80     var userManager = services.GetRequiredService<UserManager<ApplicationUser>>();
81     var context = services.GetRequiredService<ApplicationDbContext>();
82
83     // Ejecutar la lógica asíncrona dentro de un Task.Run para no bloquear el inicio. away funciona
84     // como un "dispara y olvida" para esta tarea de inicialización.
85     await Task.Run(async () => {
86         // 1. Crear Roles
87         string[] roleNames = { Roles.Admin, Roles.Technician, Roles.Employee };
88         foreach (var roleName in roleNames)
89         {
90             if (!await roleManager.RoleExistsAsync(roleName))
91             {
92                 await roleManager.CreateAsync(new IdentityRole(roleName));
93             }
94         }
95
96         // 2. Crear Admin por defecto
97         var adminEmail = "admin@system.com";
98         var adminUser = await userManager.FindByEmailAsync(adminEmail);
99         if (adminUser == null)
100         {
101             adminUser = new ApplicationUser { UserName = adminEmail, Email = adminEmail, EmailConfirmed = true };
102             await userManager.CreateAsync(adminUser, "P@ssword123!");
103             await userManager.AddToRoleAsync(adminUser, Roles.Admin);
104         }
105
106         // 3. Crear Categorías
107         if (!context.Categories.Any())
108         {
109             context.Categories.AddRange(
110                 new Category { Name = "Hardware", Description = "Physical devices" },
111                 new Category { Name = "Software", Description = "Applications" },
112                 new Category { Name = "Network", Description = "Connectivity" }
113             );
114             await context.SaveChangesAsync();
115         }
116     });
117 }
118 // =====
119 // BLOQUE DE SEEDING (FIN)
120 // =====
121
122 app.Run();
123
```

3. Restricción de UI con AuthorizeView

Ahora que tenemos roles, queremos que el menú de navegación sea inteligente, por ejemplo, si no eres Admin, no deberías ver el enlace a la gestión de técnicos.

Editar el Archivo: /Components/Layout/NavMenu.razor. Reemplaza todo el contenido del por este código:

```
1  @using Microsoft.AspNetCore.Components.Authorization
2  @using SupportTicketsSystem2026.Data
3
4  <div class="top-row ps-3 navbar navbar-dark">
5      <div class="container-fluid">
6          <a class="navbar-brand" href="">Ticket System</a>
7      </div>
8  </div>
9
10 <input type="checkbox" title="Navigation menu" class="navbar-toggler" />
11
12 <div class="nav-scrollable" onclick="document.querySelector('.navbar-toggler').click()">
13     <nav class="flex-column">
14
15         @* --- SECCIÓN 1: Enlaces Públicos --- *@
16         <div class="nav-item px-3">
17             <NavLink class="nav-link" href="" Match="NavLinkMatch.All">
18                 <span class="bi bi-house-door-fill" aria-hidden="true"></span> Home
19             </NavLink>
20         </div>
21
22         @* --- SECCIÓN 2: Enlaces para Usuarios Autenticados --- *@
23         <AuthorizeView>
24             <Authorized>
25                 <div class="nav-item px-3">
26                     <NavLink class="nav-link" href="tickets/create">
27                         <span class="bi bi-plus-square-fill" aria-hidden="true"></span> New Ticket
28                     </NavLink>
29                 </div>
30
31                 @* --- SECCIÓN 3: Solo para Administradores --- *@
32                 <AuthorizeView Roles="@Roles.Admin" Context="adminContext">
33                     <div class="nav-item px-3">
34                         <NavLink class="nav-link" href="admin/dashboard">
35                             <span class="bi bi-shield-lock-fill" aria-hidden="true"></span> Admin Panel
36                         </NavLink>
37                     </div>
38                 </AuthorizeView>
39
40                 @* --- SECCIÓN 4: Gestión de Cuenta (Logout) --- *@
41                 <div class="nav-item px-3">
42                     <form action="Account/Logout" method="post">
43                         <AntiforgeryToken />
44                         <input type="hidden" name="ReturnUrl" value="" />
45                         <button type="submit">
46                             <span class="bi bi-arrow-bar-left-nav-menu" aria-hidden="true"></span>
47                             Logout (@context.User.Identity?.Name)
48                         </button>
49                     </form>
50                 </div>
51             </Authorized>
52
53             @* --- SECCIÓN 5: Solo para Invitados (Login/Register) --- *@
54             <NotAuthorized>
55                 <div class="nav-item px-3">
56                     <NavLink class="nav-link" href="Account/Register">
57                         <span class="bi bi-person-badge-nav-menu" aria-hidden="true"></span> Register
58                     </NavLink>
59                 </div>
60                 <div class="nav-item px-3">
61                     <NavLink class="nav-link" href="Account/Login">
62                         <span class="bi bi-person-badge-nav-menu" aria-hidden="true"></span> Login
63                     </NavLink>
64                 </div>
65             </NotAuthorized>
66         </AuthorizeView>
67     </nav>
68 </div>
```

- **Contexto anidado:** Usamos Context="adminContext" en el AuthorizeView del Admin para que Blazor no se confunda con el context del usuario general.
- **Seguridad en Logout:** El Logout en Blazor Web App (unificada) requiere un form con AntiforgeryToken. Si solo pones un enlace <a>, no funcionará por protección contra ataques CSRF.
- **Bootstrap Icons:** Usamos las clases bi (como bi-shield-lock-fill) que ya vienen incluidas en la plantilla para que el menú se vea profesional.

Editar el Archivo: /Components/Layout/NavMenu.razor.css

```
103
104 .nav-item form {
105     padding: 0;
106     margin: 0;
107 }
108
109 .nav-item form button[type="submit"] {
110     color: #d7d7d7;
111     background: none;
112     border: none;
113     border-radius: 4px;
114     height: 6rem;
115     display: flex;
116     align-items: center;
117     line-height: 3rem;
118     width: 100%;
119     text-align: left;
120     padding-left: 1rem;
121     padding-right: 1rem;
122     cursor: pointer;
123     font-size: 0.9rem;
124     font-family: inherit;
125 }
126
127 .nav-item form button[type="submit"]:hover {
128     background-color: rgba(255,255,255,0.1);
129     color: white;
130 }
131
132 .nav-scrollable {
133     display: none;
134 }
```



4. Filtrado de Datos en el Servicio

La seguridad no es solo visual; es, sobre todo, de datos. El `TicketService` debe ser capaz de filtrar los tickets según quién los solicita.

Archivo: /Services/TicketService.cs (Actualización)

```
1 using Microsoft.EntityFrameworkCore;
2 using SupportTicketsSystem2026.Data;
3 using SupportTicketsSystem2026.Models;
4 using System.Security.Claims;
5
6 namespace SupportTicketsSystem2026.Services; // Recuerde que debe usar su propio namespace
7
8 // Servicio para manejar la lógica de negocio relacionada con los tickets de soporte
9 // Usando Primary Constructor de C# 14
10 public class TicketService(IDbContextFactory<ApplicationDbContext> dbFactory)
11 {
12     // --- MÓDULO 4: MÉTODOS DE CREACIÓN ---
13
14     public async Task<List<Category>> GetCategoriesAsync()
15     {
16         using var context = dbFactory.CreateDbContext();
17         return await context.Categories.ToListAsync();
18     }
19
20     public async Task CreateTicketAsync(Ticket ticket)
21     {
22         using var context = dbFactory.CreateDbContext();
23         context.Tickets.Add(ticket);
24         await context.SaveChangesAsync();
25     }
26
27     // --- MÓDULO 5: SEGURIDAD Y FILTRADO ---
28
29     public async Task<List<Ticket>> GetTicketsForUserAsync(ClaimsPrincipal user)
30     {
31         using var context = dbFactory.CreateDbContext();
32         var userId = user.FindFirst(ClaimTypes.NameIdentifier)?.Value;
33
34         var query = context.Tickets
35             .Include(t => t.Category)
36             .Include(t => t.Author)
37             .Include(t => t.Technician)
38             .AsQueryable();
39
40         if (user.IsInRole(Roles.Admin))
41         {
42             // El admin ve todo
43             return await query.ToListAsync();
44         }
45         else if (user.IsInRole(Roles.Technician))
46         {
47             return await query.Where(t => t.TechnicianId == userId || t.Status == Enums.TicketStatus.Open).ToListAsync();
48         }
49         // El empleado solo ve los tickets que él mismo creó
50         return await query.Where(t => t.AuthorId == userId).ToListAsync();
51     }
52 }
53 }
```



Resultado Esperado Final del Módulo (antes de ejecutar, realice una limpieza profunda de su base de datos).

- 1. Al iniciar la aplicación, la base de datos se poblará automáticamente con los roles en la tabla `AspNetRoles`.
- 2. Puedes registrar tres usuarios distintos y, mediante SQL o código, asignarles roles diferentes.
- 3. **Resultado visible:** Si entras como un usuario sin rol, el menú será básico. Si entras como `admin@system.com`, verás opciones avanzadas.
- 4. La lógica de "Mis Tickets" solo mostrará los registros que pertenecen legalmente al usuario logueado.

Limpieza profunda de la Base de Datos del Proyecto

Para "limpiar" la base de datos y empezar de cero con la estructura de roles y el administrador configurado, **seguir los siguientes pasos en orden estricto:**

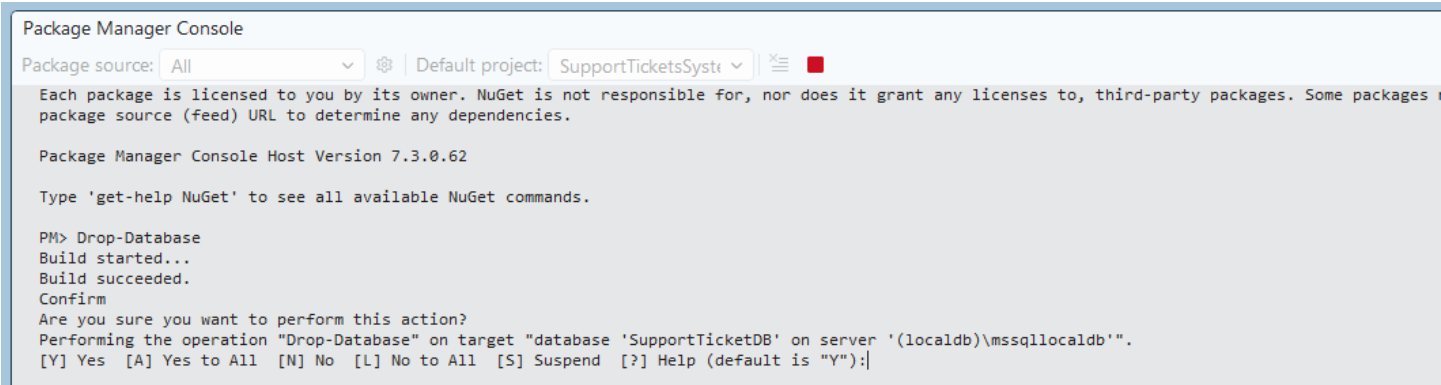
1. Limpiar el Código (Visual Studio)

- 1. **Detener la ejecución** del proyecto.
- 2. En el Explorador de Soluciones, busca la carpeta **Migrations** y **borrarla por completo** (eliminar con **cuidado** todos los archivos dentro de la carpeta).

2. Limpiar la Base de Datos (Package Manager Console)

Ejecuta los siguientes comandos uno por uno:

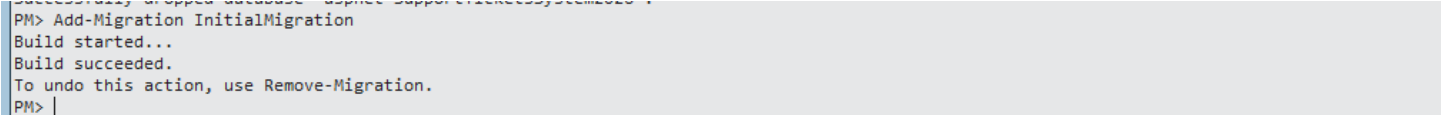
a. Borrar la base de datos física: **Drop-Database**



Escribe **Y**, y presiona **Enter** cuando pregunte si estás seguro.

- b. **Crear una ÚNICA migración limpia:** Esto creará un solo archivo que contiene todo: el esquema nuevo de .NET 10 (Passkeys) y tus tablas de Tickets.

Add-Migration InitialMigration



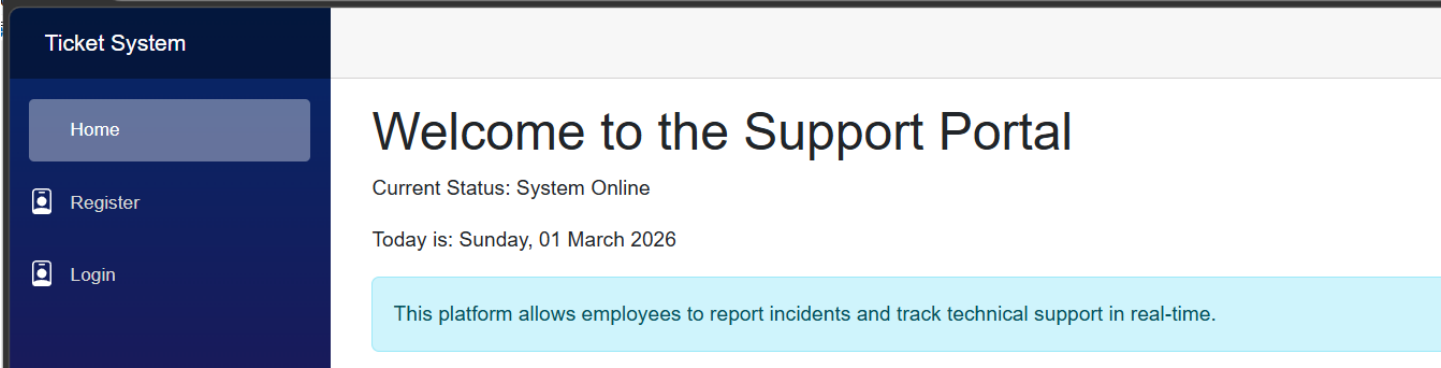
c. Aplicar a SQL Server (actualizar base de datos): **Update-Database**



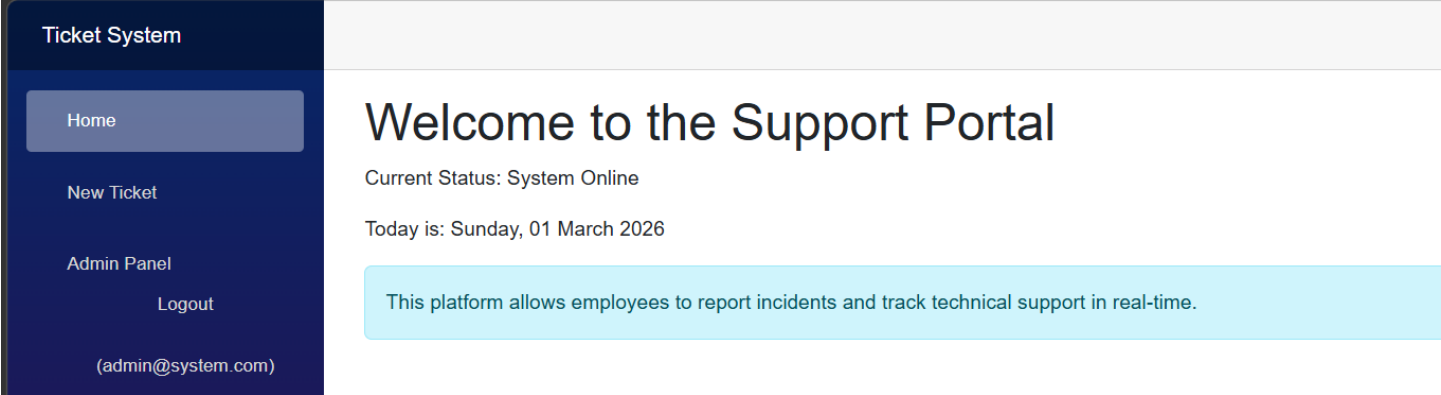
Al borrar la base de datos y las migraciones anteriores, *Add-Migration* genera un script SQL que crea las tablas directamente con la configuración y estructura actual desde el principio.

- La consola mostrará **Done** .
- Tu base de datos SupportTicketDB (o el nombre configurado), estará limpia.
- Al ejecutar la App (F5), el código de **Seeding** creará automáticamente al usuario **admin@system.com** con el rol de **Admin**.

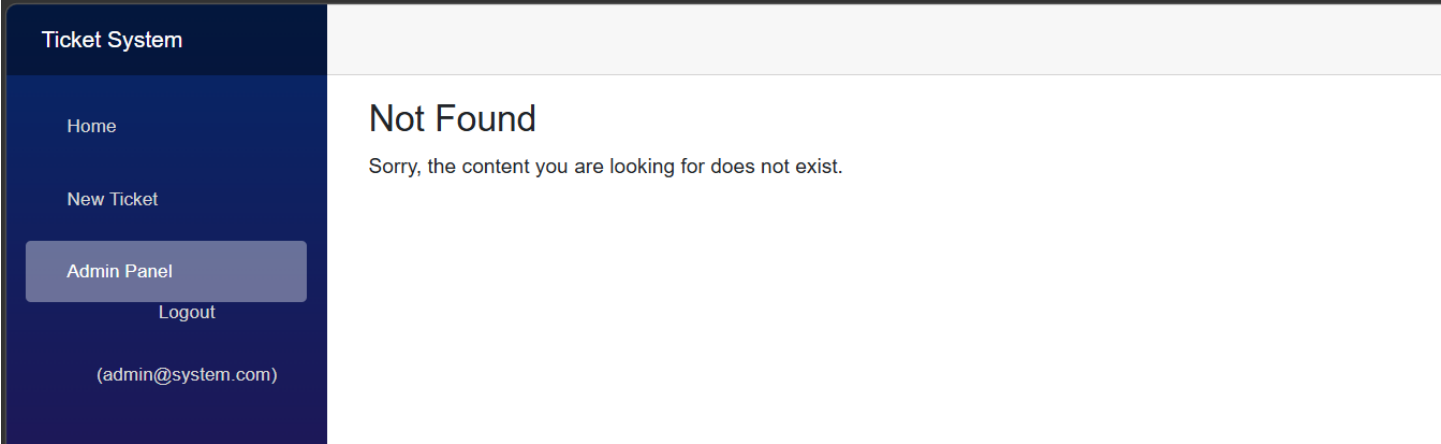
Ejecutando la App



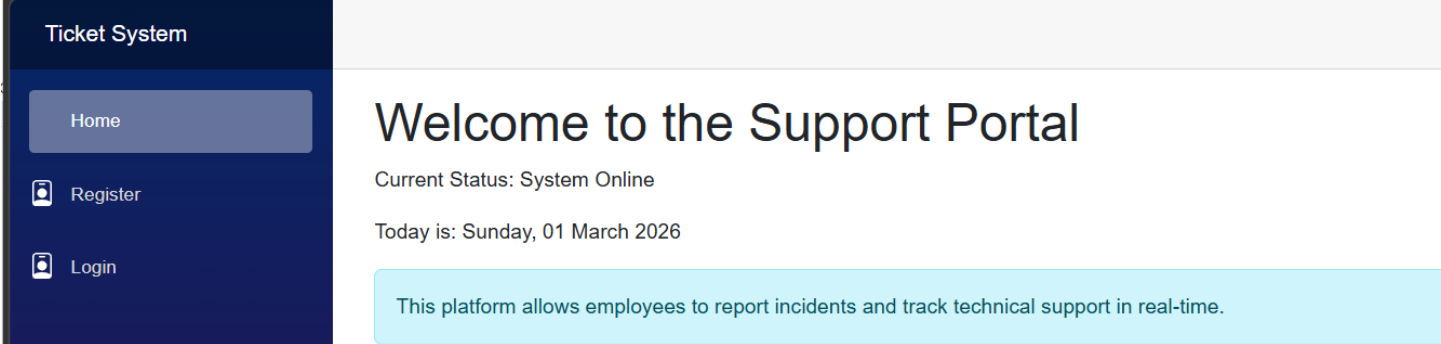
Login con el user admin: admin@system.com



Admin Panel



Logout



Registrar tres técnicos y cinco empleados.

Registrando Usuarios

Ticket System

Home

Register

Login

Register

Create a new account.

Email

empleado2@system.com

Password

Confirm Password

Register

Use another service to register.

There are no external authentication services configured. See this [article about setting up this ASP.NET application to support logging in via external services](#).

Confirmación de Registro

Ticket System

Home

Register

Login

Confirm email

Thank you for confirming your email.

Login empleado2 (sin privilegios administrativos)

Ticket System

Home

New Ticket

Logout

(empleado2@system.com)

Welcome to the Support Portal

Current Status: System Online

Today is: Sunday, 01 March 2026

This platform allows employees to report incidents and track technical support in real-time.

Verificar la base de datos. Verifique la tabla de usuarios, identifique los usuarios registrados.

Checkpoint (Agile)

- [x] Roles definidos como constantes para evitar errores.
- [x] Lógica de auto-creación de roles y usuario admin.
- [x] Uso de `AuthorizeView` para componentes visuales.
- [x] Implementación de seguridad a nivel de datos en el servicio mediante `ClaimsPrincipal`.

Actualización del Repositorio GitHub (Obligatorio)

```
Developer PowerShell
+ Developer PowerShell v | [ ] [ ] [ ]
** Copyright (c) 2026 Microsoft Corporation
*****
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   SupportTicketsSystem2026/Components/Layout/NavMenu.razor
    modified:   SupportTicketsSystem2026/Components/Layout/NavMenu.razor.css
    deleted:    SupportTicketsSystem2026/Data/Migrations/00000000000000_CreateIdentitySchema.Designer.cs
    deleted:    SupportTicketsSystem2026/Data/Migrations/00000000000000_CreateIdentitySchema.cs
    deleted:    SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.Designer.cs
    deleted:    SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.cs
    deleted:    SupportTicketsSystem2026/Data/Migrations/ApplicationDbContextModelSnapshot.cs
    modified:   SupportTicketsSystem2026/Program.cs
    modified:   SupportTicketsSystem2026/Services/TicketService.cs
    modified:   SupportTicketsSystem2026/SupportTicketsSystem2026.csproj

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    SupportTicketsSystem2026/Migrations/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSvstem2026>

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Mdulo 5:Implementando la logica de los roles jerárquicos y el acceso a los datos según la identidad del usuario."
[main 5510d51] Mdulo 5:Implementando la logica de los roles jerárquicos y el acceso a los datos según la identidad del usuario.
10 files changed, 258 insertions(+), 551 deletions(-)
delete mode 100644 SupportTicketsSystem2026/Data/Migrations/00000000000000_CreateIdentitySchema.Designer.cs
delete mode 100644 SupportTicketsSystem2026/Data/Migrations/20260319043941_AddTicketSystemTables.cs
rename SupportTicketsSystem2026/{Data/Migrations/20260319043941_AddTicketSystemTables.Designer.cs => Migrations/20260322182907_InitialMigration.Designer.cs} (99%)
rename SupportTicketsSystem2026/{Data/Migrations/00000000000000_CreateIdentitySchema.cs => Migrations/20260322182907_InitialMigration.cs} (67%)
rename SupportTicketsSystem2026/{Data => }/Migrations/ApplicationDbContextModelSnapshot.cs (100%)
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 33, done.
Counting objects: 100% (33/33), done.
Delta compression using up to 24 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (22/22), 7.55 KiB | 2.52 MiB/s, done.
Total 22 (delta 13), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (13/13), completed with 9 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
 843ec82..5510d51  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```